

# Analiza algoritmilor

---

## CAPITOLUL IV

# Cuprins

---

Introducere

Analiza algoritmilor

Notății asimptotice

Aprecierea timpului de execuție

Aprecierea costului de memorie

Profilarea unui algoritm

Concluzii

Exerciții

# Introducere

---

Termenul **algorithm** provine de la numele autorului persan Abu Ja'far Mohamed ibn Musa **Al Khowarismi** care în 825 a redactat un tratat de matematică în care prezenta pentru prima dată metode de rezolvare generice pentru anumite categorii de probleme

**Algorithm** = "Ansamblu de simboluri folosite în matematică și în logică, permițând găsirea în mod mecanic (prin calcul) a unor rezultate." "Sucesiune de operații necesare în rezolvarea unei probleme oarecare" (DEX09)

# Introducere

---

Un **algoritm** reprezintă o **metodă** (sau o rețetă) pentru a obține un rezultat dorit

Un algoritm constă din:

- Un **set de date inițiale** care abstractizează contextul problemei de rezolvat
- Un **set de relații de transformare** care sunt operate pe baza unor reguli al căror conținut și a căror succesiune reprezintă însăși substanța algoritmului
- Un **set de rezultate** preconizate sau informații finale, care se obțin de regulă trecând printr-un șir de informații intermediare

# Introducere

---

**Proprietățile** unui algoritm:

- **Generalitate** – un algoritm nu rezolvă doar o anumită problemă ci o clasă generică de probleme de același tip
- **Finitudine** – informația finală se obține din cea inițială trecând printr-un număr finit de transformări
- **Unicitate** – transformările și ordinea în care ele se aplică sunt univoc determinate de regulile algoritmului

**Consecință:** Ori de câte ori se aplică același algoritm asupra aceluiași set de date inițiale se obțin aceleași rezultate

# Introducere

---

**Scopul** capitolului este acela de a prezenta analiza performanței algoritmilor

Un important pas în analiza algoritmilor este de a determina **resursele** necesare pentru rularea algoritmului (resurse de timp și spațiu)

Un alt aspect important este de **a compara** mai mulți algoritmi în termeni de eficiență

# Analiza algoritmilor

---

## La ce servește analiza algoritmilor?

Permite precizarea **predictivă** a comportamentului algoritmilor

Prin analiză pot fi **comparați** diferiți algoritmi

## Cum putem compara doi algoritmi?

**Empiric**, implementând ambii algoritmi pe un sistem de calcul, rulând programele cu aceleași seturi de date de intrare, măsurând resursele (de timp și spațiu) utilizate și comparând rezultatele

Printr-o **analiză asimptotică**

# Analiza algoritmilor

---

## **Dezavantajele** folosirii metodei **empirice**:

- Efortul investit în implementarea unor algoritmi care nu vor fi folosiți
- Pot apărea diferențe de performanță din modul cum este scris codul, independente de diferențele de performanță ale algoritmilor
- Seturile de date de intrare pot favoriza un algoritm în detrimentul altuia
- Se poate ca niciunul din algoritmii comparați să nu se încadreze în bugetul de resurse

# Analiza algoritmilor

---

Toate acestea se pot evita folosind **analiza asimptotică**

Analiza asimptotică determină eficiența unui algoritm pentru un set numeros de date de intrare

Este de fapt o metodă de **estimare**

# Analiza algoritmilor

---

O **resursă** critică pentru un program este **timpul de rulare**

În practică nu putem să ne axăm exclusiv pe această resursă, ci trebuie să ținem cont și de spațiul de memorie (cea operativă și cea de stocare)

Există o serie de factori care influențează timpii de rulare (frecvența CPU, limbajul de programare, mediul de compilare și rulare, etc.), cu toate acestea ei nu sunt relevanți pentru compararea performanței algoritmilor

# Analiza algoritmilor

---

Una din **considerațiile de bază** pentru analiza performanței unui algoritm o reprezintă numărul de operații de bază necesare algoritmului pentru a procesa un set de date de intrare de o dimensiune dată.

Analiza algoritmilor se bazează de regulă pe **ipoteze**:

- Sistemele de calcul sunt considerate convenționale, adică ele execută câte o singură instrucțiune la un moment dat
- Timpul total de execuție al algoritmului rezultă din însumarea timpilor instrucțiunilor individuale care îl alcătuiesc

# Analiza algoritmilor

---

Exemplul 1:

Considerăm un algoritm simplu de găsim a valorii maxime într-un tablou unidimensional:

```
// Returneaza pozitia valorii maxime din vectorul "A" de dimensiune "n"
int largest(int A[], int n) {
    int index_max = 0; // in variabila se retine pozitia valorii maxime
    for (int i=1; i<n; i++) // pentru fiecare element din vector
        if (A[index_max] < A[i]) // daca A[i] este mai mare
            index_max= i; // retinem pozitia
    return index_max; // returneaza pozitia valorii maxime
}
```

# Analiza algoritmilor

---

În cazul precedent dimensiunea datelor de intrare este  $n$ , numărul de elemente din tablou

Timpul de rulare nu depinde de valoarea maximă sau de valorile din tablou ci de dimensiunea acestuia

Asfel putem exprima timpul de rulare  $T$  ca o funcție de  $n$  ( $T(n)$ )

Dacă considerăm constant timpul necesar unei comparații, atunci timpul total este:

$$T(n) = cn$$

# Analiza algoritmilor

---

Exemplul 2:

Considerăm o funcție care returnează prima valoare dintr-un tablou:

```
int first(int A[], int n) {  
    return A[0];           // returneaza prima valoare  
}
```

În acest caz timpul de rulare nu mai depinde de dimensiunea tabloului ci este constant

$$T(n) = c$$

# Analiza algoritmilor

---

Exemplul 3:

Considerăm următoarea funcție:

```
int summ(int A[], int n) {  
    sum = 0;  
    for (i=0; i<n; i++)  
        for (j=0; j<n; j++)  
            sum+=A[j];  
    return sum;  
}
```

Considerând că timpul necesar pentru o operație de adunare este constant, timpul de rulare a funcției precedente este:

$$T(n) = cn^2$$

# Analiza algoritmilor

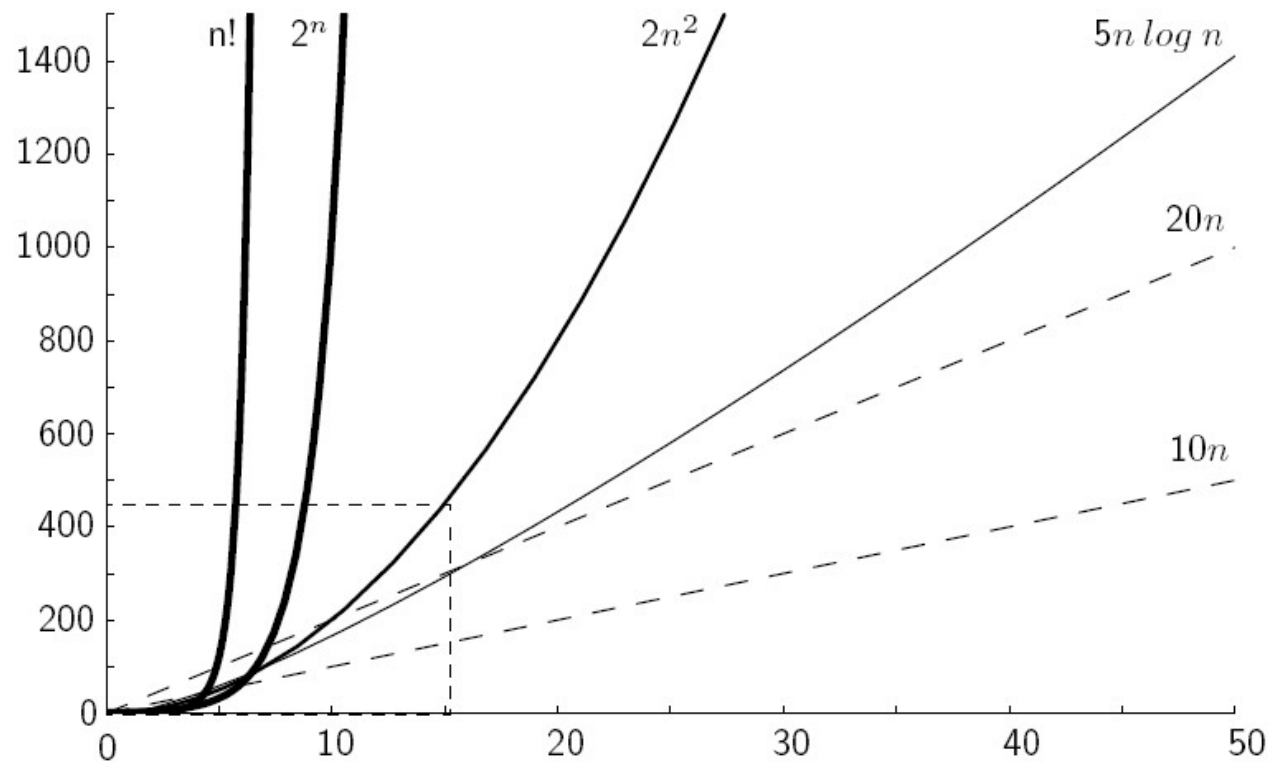
---

**Rata de creștere a timpului de rulare** al unui algoritm reprezintă costul de timp în funcție de dimensiunea datelor de intrare

Funcția de timp poate fi constantă, poate crește liniar, pătratic, logaritm etc.

Rata de creștere a timpului de rulare ne ajută să comparăm diferiți algoritmi

# Analiza algoritmilor



# Analiza algoritmilor

---

Observăm că pentru o valoare de intrare destul de mare valorile timpilor de execuție sunt influențate mai mult de forma funcției de creștere (pe ordinul de mărime) decât de constantele care apar în funcții

Astfel, în analiza algoritmilor ne axăm pe determinarea ordinului de mărime al timpului de execuție al unui program, nu pe aflarea timpilor exacti de execuție

Ordinul de mărime se determină prin studiul **eficienței asimptotice**

# Analiza algoritmilor

---

Cel mai **favorabil**, cel mai **defavorabil** caz și cazul **mediu**

- Există și funcții pentru care nu avem o variație a dimensiunii datelor de intrare și totuși timpul de rulare diferă, de data aceasta în funcție de valorile datelor de intrare (ex. factorial ( $n$ ))
- Ex: În cazul unei căutări secvențiale într-un tablou, elementul căutat se poate afla pe prima poziție (cel mai favorabil caz), pe ultima poziție (cel mai defavorabil caz). În medie algoritmul de căutare secvențială face  $n/2$  verificări (acesta este considerat cazul mediu)

# Analiza algoritmilor

---

Când analizăm un algoritm ce caz ar trebui luat în considerare?

- De obicei se consideră cel mai **defavorabil** caz
- În unele situații se consideră și cazul mediu

**Avantajul** analizării cazului cel mai **defavorabil**

- Determinăm o limită superioară pentru performanța acestuia (în toate celelalte cazuri, algoritmul se comportă mai bine decât pentru cazul cel mai defavorabil). Are o deosebită aplicabilitate în sisteme timp real (ex. control trafic aerian, sisteme anti-racheta).

**Dezavantaj:**

- Nu constituie o analiză reprezentativă pentru cazurile în care costurile de execuție trebuie agregate (ex. însumate)

# Analiza algoritmilor

---

## **Avantajul** analizării cazului **mediu**

- Constituie o analiză reprezentativă pentru cazurile în care costurile de execuție trebuie agregate (ex. însumate)

## **Dezavantaje:**

- Nu este întotdeauna posibilă determinarea cazului mediu
- Distribuția datelor joacă un rol esențial în determinarea cazului mediu

# Notății asimptotice

---

În **studiul eficienței asimptotice**, ne interesează cu precădere **limita** la care tinde timpul de execuție al algoritmului odată cu creșterea nelimitată a dimensiunii intrării

În **analiza asimptotică** se folosesc o serie de notații

# Notății asimptotice

---

## Notăția $\Theta$ (teta)

### Definiție:

Fiind dată o funcție  $g(n)$ , prin  $\Theta(g(n))$  se desemnează o mulțime de funcții definite astfel:

$$\begin{aligned} &\Theta(g(n)) \\ &= \{f(n) : \exists \text{ constantele pozitive } c_1, c_2 \text{ și } n_0 \text{ astfel încât } 0 \leq c_1 \cdot g(n) \leq f(n) \\ &\leq c_2 \cdot g(n), \text{ pentru } \forall n \geq n_0\} \end{aligned}$$

# Notatii asimptotice

---

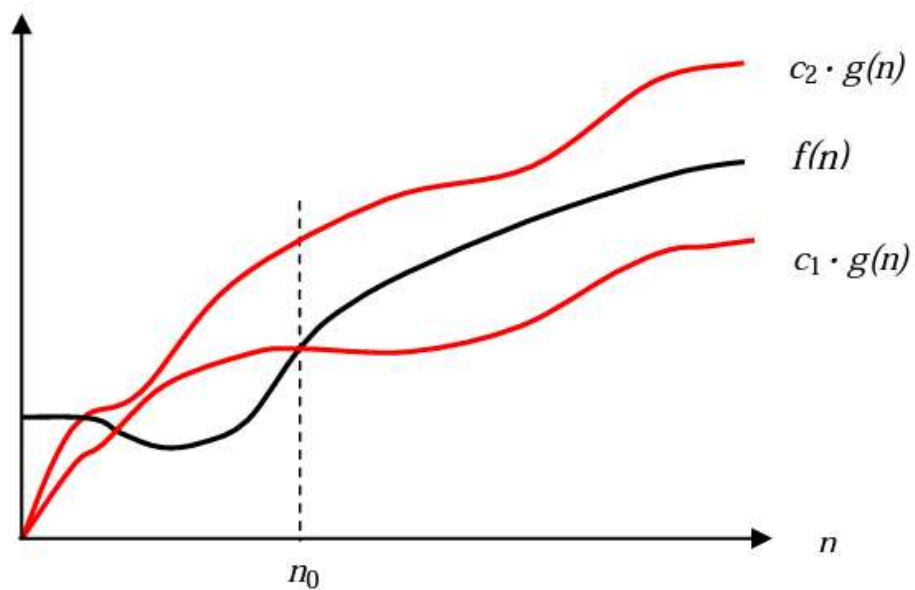
Se spune că o funcție  $f(n)$  aparține mulțimii  $\Theta(g(n))$ , dacă există constantele pozitive  $c_1$  și  $c_2$ , astfel încât ea poate fi "cuprinsă" între  $c_1g(n)$  și  $c_2g(n)$ , pentru un  $n$  suficient de mare

Cu alte cuvinte funcția  $f(n)$  este mărginită inferior de  $c_1g(n)$  și superior de  $c_2g(n)$

Se spune că  $g(n)$  este o margine **asimptotică strânsă** ("asymptotically tight bound") a lui  $f(n)$ .

# Notații asimptotice

---



$$f(n) = \Theta(g(n))$$

v 2.0/2024

# Notații asimptotice

---

În practică, determinarea lui  $\Theta$  în cazul unei expresii polinomiale, se realizează luând în considerare termenii de ordinul cel mai mare și neglijând restul termenilor

Exemplu:

$$\frac{1}{2} \cdot n^2 - 3 \cdot n = \Theta(n^2)$$

Constantele  $c_1$ ,  $c_2$  și  $n_0$  se determină astfel încât, pentru orice  $n \geq n_0$  să fie valabilă relația:

$$c_1 \cdot n^2 \leq \frac{1}{2} \cdot n^2 - 3 \cdot n \leq c_2 \cdot n^2$$

Un exemplu de valori sunt  $c_1=1/4$ ,  $c_2=1/2$  și  $n_0=12$

Egalând aceste valori se poate verifica relația

# Notatii asimptotice

---

$$\frac{1}{2} \cdot n^2 - 3 \cdot n = \Theta(n^2)$$

Exemplu de determinare  $c_1$ ,  $c_2$  și  $n_0$  pentru:

$$c_1 \cdot n^2 \leq \frac{1}{2} \cdot n^2 - 3 \cdot n \leq c_2 \cdot n^2$$

Determinăm  $c_2$ :

$$\frac{1}{2} \cdot n^2 - 3 \cdot n \leq c_2 \cdot n^2 \mid : n^2$$

$$\frac{1}{2} - \frac{3}{n} \leq c_2$$

$$\frac{1}{2} - c_2 \leq \frac{3}{n}$$

Pentru  $n > 0$ , putem alege  $c_2 = 1/2$

# Notății asimptotice

---

$$c_1 \cdot n^2 \leq \frac{1}{2} \cdot n^2 - 3 \cdot n \leq c_2 \cdot n^2$$

Determinăm  $c_1$ :

$$c_1 \cdot n^2 \leq \frac{1}{2} \cdot n^2 - 3 \cdot n \mid : n^2$$

$$c_1 \leq \frac{1}{2} - \frac{3}{n}$$

$$c_1 - \frac{1}{2} \leq -\frac{3}{n} \Rightarrow \frac{3}{n} \leq \frac{1}{2} - c_1$$

$$\text{Alegând } c_1 < 1/2, \text{ de exemplu } c_1 = 1/4 \quad \Rightarrow \quad \frac{3}{n} \leq \frac{1}{2} - \frac{1}{4} \quad \Rightarrow \quad \frac{3}{n} \leq \frac{1}{4} \mid * 4$$

$$\Rightarrow \frac{12}{n} \leq 1 \quad \Rightarrow n \geq 12 \quad \Rightarrow \quad n_0 = 12$$

# Notatii asimptotice

---

Deoarece o funcție polinomială de grad zero este o **constantă**, despre orice funcție constantă se poate spune ca este  $\Theta(n^0)$  sau  $\Theta(1)$ .

Deși acesta este un abuz de interpretare (deoarece  $n$  nu tinde la infinit), prin **convenție**  $\Theta(1)$  desemnează fie o **constantă** fie o **funcție constantă** în raport cu o variabilă.

# Notatii asimptotice

---

## Notatia O ( O mare)

Notatia O desemnează **marginea asimptotică superioară** a unei funcții. Pentru o funcție dată  $f(n)$ , se definește  $O(g(n))$  ca și mulțimea de funcții:

$$O(g(n)) = \{f(n) : \exists \text{ constantele pozitive } c \text{ și } n_0 \text{ astfel încât } 0 \leq f(n) \leq c \cdot g(n), \text{ pentru } \forall n \geq n_0\}$$

# Notatii asimptotice

---

## Notatia O:

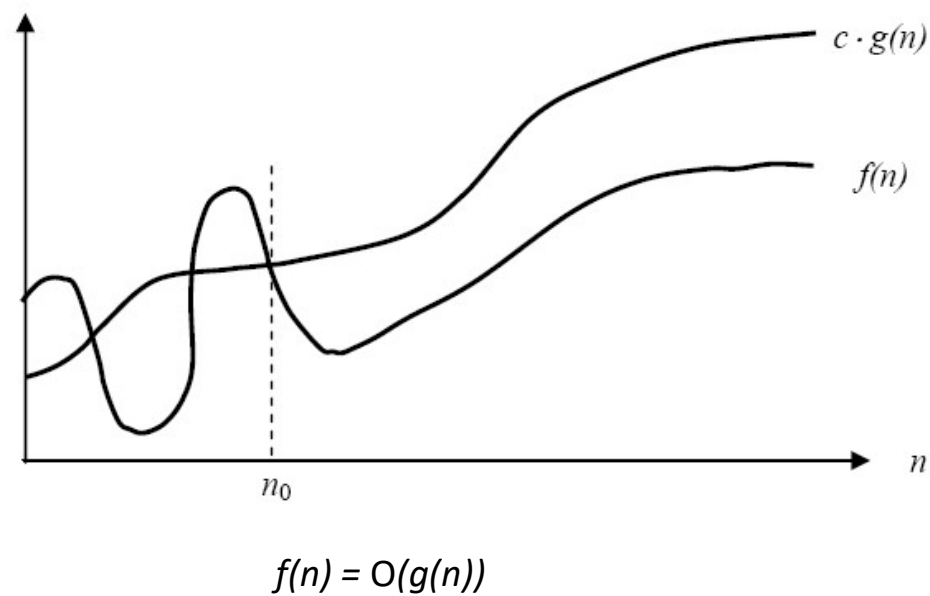
- se utilizează pentru a desemna o **margină superioară** a unei funcții în interiorul unui factor constant
- reprezintă o **limită superioară de creștere** a unei funcții

Faptul că  $f(n)$  este  $\Theta(g(n))$  implică că  $f(n) = O(g(n))$  deoarece notația  $\Theta$  este mai puternică decât notația  $O$ . Formal acest lucru se precizează prin relația:

$$\Theta(g(n)) \subseteq O(g(n))$$

# Notații asimptotice

---



# Notatii asimptotice

---

Deoarece s-a demonstrat faptul că orice funcție pătratică  $a \cdot n^2 + b \cdot n + c$ ,  $a > 0$  este  $\Theta(n^2)$ , rezultă ca această funcție este implicit și  $O(n^2)$

**Notatia O** este de obicei cea mai utilizată în aprecierea **timpului de execuție al algoritmilor** respectiv a **performanței** acestora

Uneori ea poate fi estimată direct din **inspectarea structurii algoritmului**, spre exemplu existența unei bucle duble conduce de regulă, la o margine de ordinul  $O(n^2)$

# Notatii asimptotice

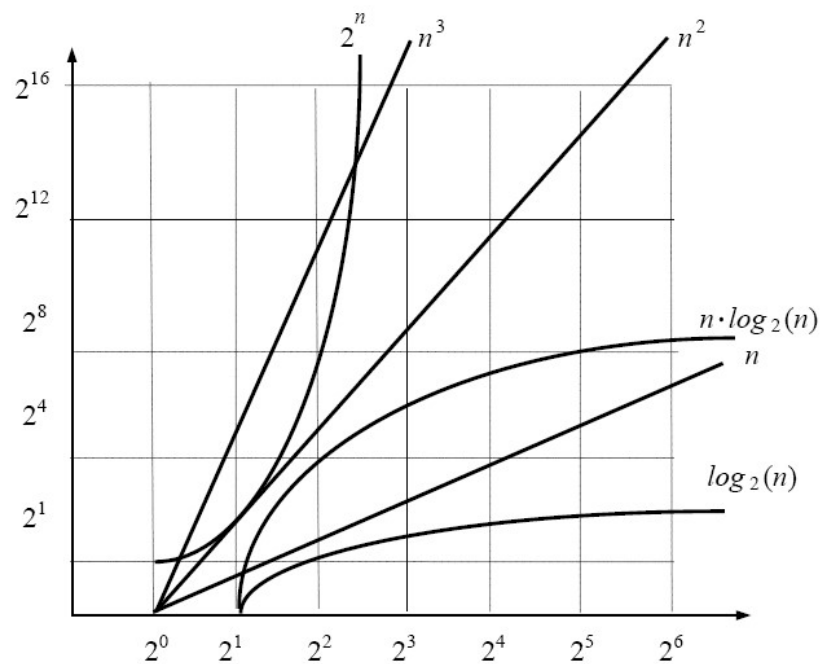
---

Deoarece notația  $O$  descrie o **margină superioară** și atunci când este utilizată, ea mărginește **cazul cel mai defavorabil** de execuție al unui algoritm

Prin implicație, ea **mărginește superior** comportamentul algoritmului în aceeași măsură pentru **orice** altă intrare

Propoziția **nu** este valabilă și pentru  $\Theta$

# Notații asimptotice



Ordine de mărime ale notației O

# Notatii asimptotice

---

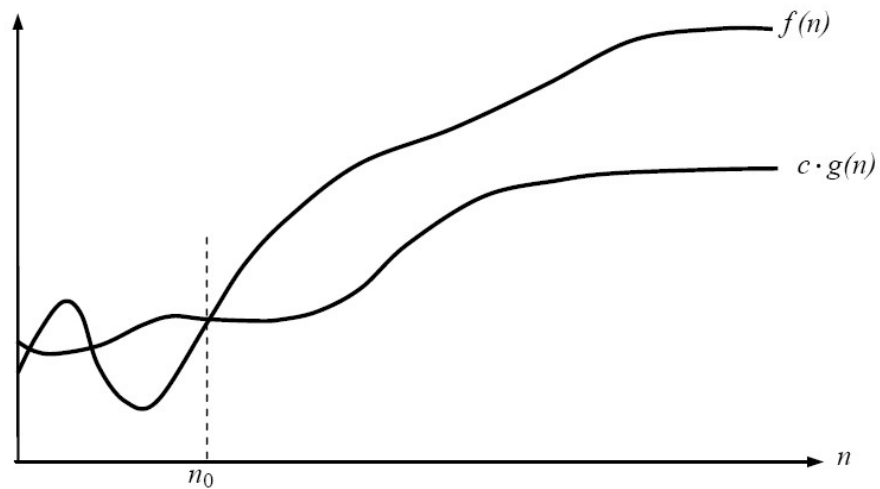
## Notatia $\Omega$ (Omega mare)

- Notatia  $\Omega$  precizează o **margină asimptotică inferioară**. Pentru o funcție dată  $f(n)$ , prin  $\Omega(g(n))$  se precizează mulțimea funcțiilor:

$$\Omega(g(n)) = \{f(n): \exists \text{ constantele pozitive } c \text{ și } n_0 \text{ astfel încât } 0 \leq c \cdot g(n) \leq f(n), \text{ pentru } \forall n \geq n_0\}$$

# Notatii asimptotice

---



$$f(n) = \Omega(g(n))$$

# Notatii asimptotice

---

Teoremă: Pentru oricare două funcții  $f(n)$  și  $g(n)$ ,  $f(n) = (g(n))\Theta$  dacă și numai dacă  $f(n) = O(g(n))$  și  $f(n) = \Omega(g(n))$

Deoarece notația  $\Omega$  descrie o **limită inferioară**, atunci când este utilizată pentru a mărgini cazul cel mai favorabil de execuție al unui algoritm, prin implicație ea **mărginește inferior** orice intrare arbitrară a algoritmului.

# Notății asimptotice

---

Notăția  $o$  (o mic)

- Marginea asimptotică superioară desemnată prin notația  $O$ , poate fi din punct de vedere asimptotic strânsă sau lejeră (laxă).
- Pentru desemnarea unei **marginii asimptotice lejere** se utilizează notația  $o$  (o mic).

Principala diferență dintre notațiile  $O$  și  $o$  rezidă în faptul că în cazul  $f(n) = O(g(n))$ , marginea  $0 \leq f(n) \leq c \cdot g(n)$  este valabilă pentru **anumite constante**  $c > 0$ , în timp ce  $f(n) = o(g(n))$ , marginea  $0 \leq f(n) < c \cdot g(n)$  este valabilă pentru **orice constantă**  $c > 0$

În notația  $o$ , funcția  $f(n)$  devine nesemnificativă în raport cu  $g(n)$  când  $n$  tinde la infinit

$$f(n) = o(g(n)) \text{ implică } \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$$

# Notății asimptotice

---

Notăția  $\omega$  (omega mic)

- Prin analogie, notația  $\omega$  este pentru notația  $\Omega$  ceea ce este  $O$  pentru  $O$ .
- Cu alte cuvinte notația  $\omega$  precizează o **margină asimptotică inferioară lejeră**.
- În relația  $f(n) = \omega(g(n))$ ,  $f(n)$  devine arbitrară în raport cu  $g(n)$  atunci când  $n$  tinde la infinit.

$$f(n) = \omega(g(n)) \text{ implică } \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$$

# Notații asimptotice

---

**Proprietăți** ale notațiilor asimptotice:

**Tranzitivitate:**

- $f(n) = \Theta(g(n))$  și  $g(n) = \Theta(h(n))$  implică  $f(n) = \Theta(h(n))$
- $f(n) = O(g(n))$  și  $g(n) = O(h(n))$  implică  $f(n) = O(h(n))$
- $f(n) = \Omega(g(n))$  și  $g(n) = \Omega(h(n))$  implică  $f(n) = \Omega(h(n))$
- $f(n) = o(g(n))$  și  $g(n) = o(h(n))$  implică  $f(n) = o(h(n))$
- $f(n) = \omega(g(n))$  și  $g(n) = \omega(h(n))$  implică  $f(n) = \omega(h(n))$

# Notatii asimptotice

---

**Proprietăți** ale notațiilor asimptotice (continuare):

**Reflexivitate:**

- $f(n) = \Theta(f(n))$
- $f(n) = O(f(n))$
- $f(n) = \Omega(f(n))$

# Notății asimptotice

---

**Proprietăți** ale notațiilor asimptotice (continuare):

**Simetrie:**

- $f(n) = \Theta(g(n))$  dacă și numai dacă  $g(n) = \Theta(f(n))$

**Simetrie transpusă**

- $f(n) = O(g(n))$  dacă și numai dacă  $g(n) = \Omega(f(n))$
- $f(n) = o(g(n))$  dacă și numai dacă  $g(n) = \omega(f(n))$

# Notatii asimptotice

---

## Reguli de simplificare:

1. Dacă  $f(n)$  este în  $O(g(n))$  și  $g(n)$  este în  $O(h(n))$ , atunci  $f(n)$  este în  $O(h(n))$
2. Dacă  $f(n)$  este în  $O(kg(n))$ , pentru orice constantă  $k > 0$ , atunci  $f(n)$  este în  $O(g(n))$
3. Dacă  $f_1(n)$  este în  $O(g_1(n))$  și  $f_2(n)$  este în  $O(g_2(n))$ , atunci  $f_1(n) + f_2(n)$  este în  $O(\max(g_1(n), g_2(n)))$
4. Dacă  $f_1(n)$  este în  $O(g_1(n))$  și  $f_2(n)$  este în  $O(g_2(n))$ , atunci  $f_1(n) \cdot f_2(n)$  este în  $O(g_1(n) \cdot g_2(n))$

# Notatii asimptotice

---

**Prima regulă** spune că dacă o funcție  $g(n)$  reprezintă o limită superioară pentru funcția de cost  $f(n)$ , atunci orice limită superioară a funcției  $g(n)$  reprezintă o limită superioară și pentru  $f(n)$

O proprietate similară este valabilă și pentru notația  $\Omega$ : dacă o funcție  $g(n)$  reprezintă o limită inferioară pentru funcția de cost  $f(n)$ , atunci orice limită inferioară a funcției  $g(n)$  reprezintă o limită inferioară și pentru  $f(n)$

**A doua regulă** spune că putem ignora orice constantă multiplicativă din ecuație când folosim notația  $O$ .

Regula este valabilă și pentru notațiile  $\Omega$  și  $\Theta$

**A treia regulă** spune că dacă avem două părți de program care rulează secvențial, trebuie să luăm în considerare partea cea mai costisitoare

Regula este valabilă și pentru notațiile  $\Omega$  și  $\Theta$

# Notații asimptotice

---

**A patra regulă** este folosită pentru a analiza bucle simple în program. Dacă o acțiune se repetă de un număr de ori și acțiunea are același cost de fiecare dată, atunci costul total este costul unei acțiuni multiplicat cu numărul de repetiții ale acelei acțiuni.

Regula este valabilă și pentru notațiile  $\Omega$  și  $\Theta$

# Notatii asimptotice

---

## Compararea funcțiilor

- Fiind date două funcții  $f(n)$  și  $g(n)$ , ale căror rată de creștere este exprimată sub formă de ecuații. Am vrea să determinăm care din ele crește mai repede.
- Putem afla care din ele are rata de creștere mai mare aflând următoarea limită

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

- Dacă limita tinde la infinit, atunci  $f(n)$  este în  $\Omega(g(n))$ , deoarece  $f(n)$  crește mai repede
- Dacă limita tinde la zero, atunci  $f(n)$  este în  $O(g(n))$ , deoarece  $g(n)$  crește mai repede
- Dacă limita tinde spre o constantă, alta decât zero, atunci  $f(n) = \Theta(g(n))$ , deoarece ambele cresc cu aceeași rată

# Aprecierea timpului de execuție

---

Cu ajutorul notației **O** (O mare) se poate aprecia **ordinul timpului de execuție** al unui algoritm

Se face sublinierea că aprecierea se referă atât la ordinul timpului de execuție al unui **algoritm abstract** cât și cel al unui **program** real rezultat din implementarea respectivului algoritm.

**Timpul efectiv** de execuție al unui program, **depinde** de mai mulți factori cum ar fi:

- (1) **Dimensiunea și natura** datelor de intrare.
- (2) **Caracteristicile sistemului de calcul** pe care se rulează programul.
- (3) **Eficiența codului** produs de compilator.

# Aprecierea timpului de execuție

---

Notăția  $O$  mare permite eliminarea factorilor care nu pot fi controlați, cum ar fi spre exemplu (2) și (3) enumerați mai sus, concentrându-se asupra comportării algoritmului independent de program.

În general un algoritm a cărui complexitate temporală este  $O(n^2)$  va rula ca și program în  $O(n^2)$  unități de timp indiferent de limbajul sau sistemul de calcul utilizat.

În aprecierea timpului de execuție se pornește de la **ipoteza simplificatoare** deja enunțată, că fiecare instrucție utilizează în medie aceeași cantitate de timp

# Aprecierea timpului de execuție

---

Instrucțiunile care **nu pot fi încadrate** în această medie de timp sunt:

- Instrucțiunea **IF**
- Secvențele **repetitive** (buclele)
- **Apelurile** de funcții.

# Aprecierea timpului de execuție

---

Presupunând pentru moment că apelurile de funcții se ignoră, se consideră **cel mai defavorabil caz** și se adoptă prin convenție următoarele **simplificări**:

- Se presupune că o instrucțiune IF va consuma întotdeauna timpul necesar **execuției ramurii celei mai lungi**, dacă nu există rațiuni contrare justificate;
- Se presupune că întotdeauna instrucțiunile din interiorul unei bucle se vor executa de **numărul maxim** de ori permis de condiția de control

# Aprecierea timpului de execuție

---

## **Ex1:**

Atribuirea unei valori pentru o variabilă de tip întreg

`a = b;`

Pentru că timpul pentru executarea instrucțiunii de atribuire este constant, avem  $\Theta(1) \Rightarrow O(1)$

# Aprecierea timpului de execuție

---

## Ex2:

Considerăm o buclă for:

```
int sum=0;  
for (int i =0; i<n;i++)  
    sum+=n;
```

Prima linie este  $\Theta(1)$ , implicit și  $O(1)$ .

Bucla se repetă de  $n$  ori. Durata execuției celei de-a treia linii este constantă. Din regula de estimare a buclelor  $\Rightarrow \Theta(n)$  pentru întregul cod.

# Aprecierea timpului de execuție

---

## Ex3:

Considerăm mai multe bucle:

```
sum = 0;
for (i=1; i<=n; i++)    // Prima bucla
    for (j=1; j<=i; j++)    // O bucla dubla
        sum++;
for (k=0; k<n; k++)    // A doua bucla
    A[k] = k;
```

# Aprecierea timpului de execuție

---

## Ex3:

Acest exemplu are trei secvențe:

- O atribuire
- Două bucle

Prima secvență este  $c_1 = \Theta(1)$  și ultima  $c_2 \cdot n = \Theta(n)$ , a doua secvență se calculează astfel:

- `sum++` necesită timp constant
- For-ul interior se execută de  $i$  ori  $\Rightarrow c_3 \cdot i$ , iar cel exterior de  $n$  ori
- Costul total =  $c_3 \cdot \sum_{i=1}^n i = \frac{n(n+1)}{2} = \Theta(n^2)$ , din regula de însumare avem  $\Theta(n^2)$  pentru întregul cod

# Aprecierea timpului de execuție

---

**Nu** toate buclele duble sunt  $\Theta(n^2)$

**Ex4:**

```
sum1 = 0;
for (k=1; k<=n; k*=2) // se repetă de log n ori
    for (j=1; j<=n; j++) // se repetă de n ori
        sum1++;
```

# Aprecierea timpului de execuție

---

## Ex4:

Presupunem că  $n$  este putere a lui 2.

Primul for din bucla dublă se execută de  $\log n + 1$

Al doilea for se execută de  $n$  ori

Costul total =  $\sum_{i=0}^{\log n} n = \Theta(n \log n)$

# Aprecierea timpului de execuție

---

Buclele while și do – while se analizează într-un mod asemănător cu bucla for

Pentru instrucțiunile if și switch se determină ramura ce reprezintă cazul cel mai defavorabil și se calculează timpul de execuție pentru acea ramură

# Aprecierea timpului de execuție

---

Determinarea timpului de execuție pentru o subrutină recursivă poate fi dificilă

De obicei se determină o relație de recurență pentru calcularea timpului de execuție

**Ex1:** funcția factorial

- $T(n) = T(n-1) + c$ , pentru  $n > 1$ ,  $T(1) = c$

S-a demonstrat în cursul anterior că timpul de execuție este direct proporțional cu  $n \Rightarrow \Theta(n)$

# Aprecierea timpului de execuție

---

## **Ex2:** Căutarea binară

- $T(n) = T(n/2) + c$  pentru  $n > 1$ ;

$$T(1) = c$$

$$T(2) = T(1) + c = 2c$$

$$T(3) = T(1) + c = 2c$$

$$T(4) = T(2) + c = 3c$$

...

$$T(8) = T(4) + c = 4c \Rightarrow T(k) = (\lfloor \log_2 n \rfloor + 1) c \Rightarrow O(\log n)$$

În cursurile viitoare privind recursivitatea vom discuta mai multe exemple

# Aprecierea timpului de execuție

---

## Ex3: Parametrii multipli

- Fie o figură cu P pixeli, care pot avea un cod de culoare între 0 și C-1. Să se afle numărul de pixeli de fiecare culoare, apoi să se sorteze culorile în funcție de numărul de pixeli de fiecare culoare

```
for (i=0; i<C; i++) // initializare
    count[i] = 0;
for (i=0; i<P; i++) // Parcurgere pixeli
    count[value(i)]++; /* incrementarea contorului pentru
culoarea value(i) */
sort(count, C); //functia de sortare
```

# Aprecierea timpului de execuție

---

## Ex3: Parametrii multiplii

- În exemplul anterior vectorul count are dimensiunea  $C$
- Pentru prima buclă avem  $\Theta(C)$ , pentru a doua avem  $\Theta(P)$
- Pentru funcția de sortare, costul depinde de algoritmul implementat. Să presupunem că folosim o sortare cu  $\Theta(n \log n) \Rightarrow \Theta(C \log C)$  pentru sortarea colorilor
- Aceasta ne duce la un cost total de  $\Theta(P + C \log C)$ .
- Putem să considerăm  $\Theta(C \log C)$ ? Depinde de relația între  $C$  și  $P$ , dacă  $C$  este mult mai mic,  $P$  are o influență mai mare decât  $C \log C$ . Niciuna dintre variabile nu poate fi ignorată în acest caz

# Aprecierea costului de memorie

---

Pe lângă timp, spațiul (de memorie) este o altă resursă pentru care ne interesează costul

Metodele de analiză pentru costul spațiului de memorie ocupat sunt similare cu cele pentru analiza costului de timp

În timp ce costurile de timp sunt determinate pentru un algoritm ce folosește un anumit tip de structuri de date, costurile de spațiu de memorie sunt în mod normal determinate pentru structurile de date efective

Analiza asimptotică se aplică și pentru spațiul de memorie în același mod ca pentru timpii de rulare

# Aprecierea costului de memorie

---

Ex1: Care sunt costurile de spațiu de memorie pentru reținerea a  $n$  valori întregi, dacă fiecare întreg ocupa  $c$  octeți.

Dacă fiecare întreg ocupa  $c$  octeți, atunci  $n$  întregi ocupa  $c * n$  octeți  $\Rightarrow \Theta(n)$

# Aprecierea costului de memorie

---

Proiectarea algoritmilor presupune de cele mai multe ori un compromis între spațiu și timp

Ex1: Prin compresia datelor reducem spațiul utilizat, dar creștem costul de timp prin adăugarea de timpi suplimentari pentru compresie/decompresie

Ex2: Un tabel de valori pentru o funcție reduce timpii necesari recalculării acelor valori, dar solicită spațiu pentru memorarea valorilor

În plus, accesul la spațiul de stocare extern include costuri de timp suplimentare

# Profilarea unui algoritm

---

Presupunem că un algoritm a fost conceput, implementat, testat și depanat pe un sistem de calcul țintă.

Ne interesează de regulă profilul performanței sale, adică timpii preciși de execuție ai algoritmului pentru diferite seturi de date, eventual pe diferite sisteme țintă.

Pentru aceasta sistemul de calcul țintă trebuie să fie dotat cu un ceas intern și cu funcții sistem de acces la acest ceas

# Profilarea unui algoritm

---

Se presupune un algoritm implementat în forma unui program numit Algoritm(X: Intrare,Y: Iesire)unde X este intrarea iar Y ieșirea.

Pentru a construi profilul algoritmului este necesar să fie concepute:

- (1) **Seturile de date** de intrare a căror dimensiune crește între anumite limite, pentru a studia comportamentul algoritmului în raport cu dimensiunea intrării.
- (2) Seturile de date de intrare care în principiu se referă la **cazurile extreme** de comportament.
- (3) O funcție cu ajutorul căreia poate fi construit **profilul algoritmului** în baza seturilor de date anterior amintite.

# Profilarea unui algoritm

---

Funcția Profil poate fi utilizată în mai multe scopuri funcție de obiectivele urmărite.

- (1) Evidențierea performanței intrinseci a unui algoritm precizat.
- (2) Evidențierea performanței relative a doi sau mai mulți algoritmi diferiți care îndeplinesc aceeași sarcină.
- (3) Evidențierea performanței relative a două sau mai multe sisteme de calcul.

# Concluzii

---

Facem diferența între limite superioare și limite inferioare, când nu cunoaștem cu exactitate rata de creștere a funcției cost

Folosim notația  $\Theta$ , când vrem să indicăm că nu sunt diferențe majore între ratele de creștere ale limitelor inferioară și superioară pentru o funcție cost dată

Funcțiile  $O$ ,  $\Omega$  și  $\Theta$  nu determină costurile de timp efective, ci determină limite de creștere a unei funcții de cost

# Exerciții

---

Ex1: Reprezentați grafic următoarele funcții. Pentru fiecare funcție determinați intervalul de valori ale lui  $n$ , pentru care aceasta are eficiența maximă

$$4n^2; \log_3 n; 3^n; 20n; 2; \log_3 n; n^{2/3}$$

Ex2: Rescrieți în variantă iterativă următoarea funcție:

```
function( int n ) {  
    if( n == 1 ) return;  
    for(int i = 1 ; i <= n ; i ++ )  
        for(int j = 1 ; j <= n ; j ++ )  
            printf("*");  
    function( n-3 );  
}
```

Demonstrați că  $T(n) = \Theta(n^3)$  pentru funcția anterioară

# Exerciții

---

Ex3: Determinați  $\Theta(f(n))$  pentru următoarele secvențe de cod

(a)        `a = b + c;`  
          `d = a + e;`

(b)        `sum = 0;`  
          `for (i=0; i<3; i++)`  
          `for (j=0; j<n; j++)`  
              `sum++;`

(c)        `sum=0;`  
          `for (i=0; i<n*n; i++)`  
              `sum++;`

# Exerciții

---

Ex3 (Continuare): Determinați  $\Theta(f(n))$  pentru următoarele secvențe de cod

```
(d)      for (i=0; i < n-1; i++)
        for (j=i+1; j < n; j++) {
            tmp = A[i][j];
            A[i][j] = A[j][i];
            A[j][i] = tmp;      }
```

```
(e)      sum = 0;
        for (i=1; i<=n; i++)
            for (j=1; j<=n; j*=2)
                sum++;
```

```
(f)      sum = 0;
        for (i=1; i<=n; i*=2)
            for (j=1; j<=n; j++)
                sum++;
```